

EMBEDDED SYSTEMS

Top 50 Interview Q&A

Questions + Detailed Answers

Protocols

RTOS

Memory

GPIO

Power

Security

IoT

Swipe through all 50 questions

Perfect for interview prep & revision

50

Q01

What is an embedded system?

ANSWER

A dedicated computer system built into a larger device to perform specific, real-time tasks using a microcontroller or microprocessor—e.g., a washing machine controller or ABS braking unit.

Q02

Difference between microcontroller and microprocessor?

ANSWER

A microcontroller integrates CPU, RAM, ROM, and I/O peripherals on a single chip (e.g., STM32). A microprocessor is only the CPU and needs external memory and peripherals (e.g., Intel i7).

Q03

What is RTOS? Name examples.

ANSWER

Real-Time Operating System schedules tasks with deterministic timing. Examples: FreeRTOS, Zephyr, VxWorks, QNX, ThreadX. Used when hard deadlines must be met.

Q04

Explain the concept of interrupt.

ANSWER

An interrupt is a hardware or software signal that suspends the CPU's current task, executes an Interrupt Service Routine (ISR), then resumes the original task—enabling real-time event handling.

Q05

What is a watchdog timer?

ANSWER

A hardware timer that resets the system if the software fails to periodically 'kick' (reload) it within a set interval, preventing the system from hanging in a faulty state.

Q06 Types of memory in embedded systems?

ANSWER
Flash (program storage, non-volatile), SRAM (fast working RAM, volatile), EEPROM (byte-erasable NV storage), ROM (mask-programmed, immutable), and external SDRAM/NOR flash for larger systems.

Q07 What is memory-mapped I/O?

ANSWER
Peripheral registers are placed in the same address space as RAM/ROM. The CPU accesses peripherals using normal load/store instructions to specific addresses rather than special I/O instructions.

Q08 Explain stack vs heap in embedded C.

ANSWER
Stack: auto-allocated, LIFO, used for local variables and function calls—fast but limited. Heap: dynamic allocation via malloc/free—flexible but risks fragmentation and leaks in embedded systems.

Q09 What is DMA and why use it?

ANSWER
Direct Memory Access lets a DMA controller transfer data between memory and peripherals without CPU intervention, freeing the CPU for computation while large data transfers (ADC, UART) occur simultaneously.

Q10 What is the difference between RAM and ROM?

ANSWER
RAM (Random Access Memory) is volatile—data is lost on power-off—and used for runtime data. ROM is non-volatile, retains data without power, and stores firmware/bootloaders.

Q11 Explain I2C protocol.

ANSWER
Two-wire (SDA + SCL) synchronous, half-duplex, multi-master/multi-slave bus. Uses 7-bit addressing (up to 128 devices), ACK/NACK handshaking. Speeds: 100kHz (standard), 400kHz (fast), 3.4MHz (high-speed).

Q12 Explain SPI protocol.

ANSWER
Four-wire (SCLK, MOSI, MISO, SS) synchronous, full-duplex protocol. No addressing—CS/SS line selects each slave. Speeds up to 50+ MHz. Ideal for high-bandwidth peripherals: flash, ADCs, displays.

Q13 Explain UART protocol.

ANSWER
Asynchronous, full-duplex, two-wire (TX/RX) peer-to-peer protocol. No clock line—both sides use the same baud rate. Frame: START + 5-9 data bits + optional parity + STOP bit(s).

Q14 What is CAN bus?

ANSWER
Controller Area Network is a robust, differential, multi-master serial bus widely used in automotive and industrial systems. Supports up to 1 Mbps, with priority-based arbitration and built-in error detection/correction.

Q15 What is I3C protocol?

ANSWER
Improved Inter Integrated Circuit (MIPI I3C) is backward-compatible with I2C but adds dynamic addressing, in-band interrupts, higher speeds (12.5 MHz), and lower power—ideal for sensors in mobile/IoT devices.

Q16 What is GPIO?

ANSWER
General Purpose Input/Output pins can be configured as digital input (read button/sensor) or digital output (drive LED/relay). Most MCUs support alternate functions: UART, SPI, I2C, PWM on the same pins.

Q17 Explain PWM and its uses.

ANSWER
Pulse Width Modulation generates a square wave with variable duty cycle. Used for motor speed control, LED dimming, servo positioning, and DAC emulation. Frequency and duty cycle are set by timer registers.

Q18 What is ADC and how does it work?

ANSWER
Analog-to-Digital Converter samples an analog voltage and converts it to a digital number. Resolution (8–24 bit) determines precision; sample rate determines bandwidth. Common architectures: SAR, Sigma-Delta, Flash.

Q19 What is DAC?

ANSWER
Digital-to-Analog Converter converts a digital value to a proportional analog voltage or current—used for audio output, signal generation, and reference voltage production.

Q20 What are pull-up and pull-down resistors?

ANSWER
Pull-up resistors connect a pin to VCC so it reads HIGH when floating; pull-down connects to GND for LOW. They prevent undefined states on open-drain/open-collector lines and unconnected GPIO pins.

Q21 What is a task/thread in RTOS?

ANSWER
A task (thread) is an independent unit of execution with its own stack and priority. The RTOS scheduler switches between tasks based on priority and state (running, ready, blocked, suspended).

Q22 Explain preemptive vs cooperative scheduling.

ANSWER
Preemptive: the scheduler can forcibly switch a running task when a higher-priority task becomes ready. Cooperative: a task must voluntarily yield control. Preemptive is more responsive; cooperative is simpler.

Q23 What is a semaphore?

ANSWER
A synchronization primitive with a counter. Binary semaphore (0/1) signals between tasks or from ISR to task. Counting semaphore tracks multiple resources. Used to prevent race conditions on shared data.

Q24 What is a mutex?

ANSWER
Mutual Exclusion lock: only the task that acquires it can release it. Prevents simultaneous access to shared resources. Supports priority inheritance to avoid priority inversion—unlike a binary semaphore.

Q25 What is priority inversion?

ANSWER
When a high-priority task is blocked by a low-priority task holding a shared mutex, and a medium-priority task preempts the low-priority one indefinitely. Solved with priority inheritance or priority ceiling protocols.

Q26 How do you reduce power in embedded systems?

ANSWER
Use sleep/stop/standby modes between events; lower clock frequency; turn off unused peripherals; use DMA instead of busy-wait polling; choose low-power MCUs (nRF52, STM32L series); optimize code to minimize active cycles.

Q27 What are MCU low-power modes?

ANSWER
Typical modes: Sleep (CPU halted, peripherals run), Stop (clocks gated, RAM retained), Standby (only RTC/wakeup active, RAM lost), Shutdown (lowest power, full reset on wake). Wake sources: RTC, GPIO interrupt, UART.

Q28 What is dynamic voltage and frequency scaling (DVFS)?

ANSWER
DVFS reduces CPU supply voltage and clock frequency when full performance is not needed, dramatically cutting dynamic power ($P = C \cdot V^2 \cdot f$). Common in application processors like Cortex-A series.

Q29 What is a bootloader?

ANSWER
A small program that runs before the main application. It initializes hardware, validates firmware integrity, and optionally supports firmware updates over UART/USB/CAN/Ethernet before jumping to the application.

Q30 Explain the embedded boot sequence.

ANSWER
On power-on: reset vector fetched from flash, stack pointer initialized, system clock configured, peripherals initialized (HAL/BSP), C runtime (data/bss sections initialized), then main() called.

Q31 What is firmware OTA update?

ANSWER
Over-The-Air update delivers new firmware wirelessly (Wi-Fi/BLE/Cellular). The bootloader downloads the image to a secondary flash partition, verifies its hash/signature, then swaps it as the active partition on next boot.

Q32 What is linker script used for?

ANSWER
A linker script defines memory regions (Flash, SRAM, CCM), places code/data sections (.text, .data, .bss, .stack, .heap) at correct addresses, and generates the final binary image for the target MCU.

Q33 What is JTAG?

ANSWER
Joint Test Action Group: a 4/5-wire debug and boundary-scan interface (TCK, TMS, TDI, TDO, TRST). Used to program flash, set breakpoints, read/write registers and memory without disturbing MCU operation.

Q34 What is SWD?

ANSWER
Serial Wire Debug is ARM's 2-pin (SWDIO + SWDCLK) alternative to JTAG. Same functionality—programming, breakpoints, memory access—with fewer pins. Widely used on STM32, nRF, and other Cortex-M MCUs.

Q35 How do you debug a hard fault on Cortex-M?

ANSWER
Enable fault handlers, read HFSR/CFSR/BFAR/MMFAR status registers, inspect stacked PC/LR on the task stack to find the faulting instruction. Common causes: null pointer dereference, stack overflow, unaligned access.

Q36 What is semihosting?

ANSWER
A debug mechanism that routes I/O calls (printf, file open) from the embedded target to the host PC debugger via the debug probe. Useful during development but must be removed for production as it halts the CPU.

Q37 What is a crystal oscillator vs RC oscillator?

ANSWER
Crystal oscillator uses a piezoelectric crystal for high accuracy (ppm-level), needed for USB, precise UART baud rates, and RTC. RC oscillator is internal, cheaper, less accurate (+/-1-3%), used for cost-sensitive apps.

Q38 What is clock jitter?

ANSWER
Random variation in a clock edge's timing. In high-speed systems, excessive jitter causes bit errors in serial links, sampling errors in ADCs, and timing violations in synchronous logic. Minimized with PLLs and good PCB layout.

Q39 What is a PLL in an MCU?

ANSWER
Phase-Locked Loop multiplies a low-frequency crystal reference to produce high-frequency CPU/peripheral clocks. E.g., STM32 uses 8 MHz HSE crystal × PLL to generate 72–480 MHz system clock.

Q40 What is ISO 26262?

ANSWER
ISO 26262 is the automotive functional safety standard defining ASIL (Automotive Safety Integrity Level) A–D. It mandates rigorous V&V, redundancy, and fault detection for safety-critical embedded software in vehicles.

Q41 What is secure boot?

ANSWER
Secure boot uses a hardware root-of-trust (e.g., OTP-burned public key hash) to verify each firmware stage's cryptographic signature before execution, preventing unauthorized or tampered firmware from running.

Q42 What is stack canary?

ANSWER
A random value placed between local variables and the return address at function entry. Checked before returning—if overwritten, a stack overflow is detected and the system is halted/reset, preventing exploitation.

Q43 What is MQTT?

ANSWER
Message Queuing Telemetry Transport: a lightweight publish-subscribe protocol over TCP/IP. Designed for constrained IoT devices. Broker routes messages between publishers and subscribers using topics. QoS levels 0, 1, 2.

Q44 Explain CoAP protocol.

ANSWER
Constrained Application Protocol is a RESTful protocol for IoT devices, built on UDP. Supports GET/POST/PUT/DELETE like HTTP but with a tiny binary header. Suitable for LwM2M device management over NB-IoT/6LoWPAN.

Q45 What is 6LoWPAN?

ANSWER
IPv6 over Low-Power Wireless Personal Area Networks: adapts IPv6 packets for 802.15.4 radios by compressing headers and fragmenting large packets. Enables IP connectivity for tiny constrained sensor nodes.

Q46 What does 'volatile' keyword do in C?

ANSWER
Tells the compiler the variable can change outside normal program flow (e.g., ISR, hardware register). Prevents the compiler from caching it in a register or optimizing away reads/writes. Essential for peripheral registers and ISR flags.

Q47 What is the difference between #define and const in embedded C?

ANSWER
#define is a preprocessor text substitution with no type safety. 'const' creates a typed, scoped constant that the compiler can enforce type-checking on and optionally place in flash. Prefer 'const' for type safety.

Q48 What is endianness?

ANSWER
Byte order of multi-byte values. Little-endian (x86, Cortex-M): LSB at lowest address. Big-endian (network, some DSPs): MSB at lowest address. Critical when sharing data between systems or reading hardware registers.

Q49 What is bit-banding in Cortex-M?

ANSWER
An ARM Cortex-M3/M4 feature that maps each bit of a 1MB SRAM/peripheral region to a unique 32-bit word address in a 32MB alias region, enabling atomic single-bit read/write without masking—useful in ISRs.

Q50 How do you implement a circular (ring) buffer?

ANSWER
Use a fixed-size array with head and tail indices. Producer writes at tail (tail = (tail+1) % size); consumer reads at head (head = (head+1) % size). Buffer full when (tail+1)%size == head; empty when head == tail.